

FSM_API - Advanced Unity Integration

Overview: The Ultimate State Management Experience

FSM_API: Advanced Unity Integration extends the battle-tested, high-performance [FSM_API Core Library](#) into Unity's multi-threaded environment. This package is designed for creating **Complex and Full Experiences**, establishing a comprehensive world to host "In-App Experiences" created with the Basic Integration package.

Where the basic package focuses on the game *itself* (e.g., Chess, Pac-Man), the Advanced Integration manages the *environment* (e.g., an Arcade Machine, a Character's AI, the World State), offering unparalleled control and clarity for complex projects like simulations, expansive VR/AR experiences, or enterprise applications.

Advanced vs. Basic: The Experience Model

The fundamental difference lies in how FSM processing is scheduled—we refer to this as the **Experience Model**.

Feature	Basic Integration (FSM_UnityIntegration)	Advanced Integration (FSM_UnityIntegrationAdvanced)
Core Use Case	Single "In-App Experience" (e.g., a simple game/item).	Complex, multi-layered "Full Experience" (e.g., a world/character).
Processing Groups	One default Processing Group per Unity message (Update, FixedUpdate, etc.).	Multiple, Custom Processing Groups per Unity message, fully configurable.
Capability	Simple, monolithic scheduling (all FSMs process together).	Fine-grained, decoupled scheduling (AI, Physics, UI, Gameplay Logic can be segregated and prioritized).
Compatibility	Basic Experiences can run inside an Advanced Experience.	Advanced Experiences are self-contained and would host Basic Experiences.

The Heartbeat of Logic: Integration Class

The `FSM_UnityIntegrationAdvanced` class itself is the **heartbeat** of your FSM logic within Unity's main thread. It's a Unity `MonoBehaviour` singleton designed to safely bridge the engine-agnostic core `FSM_API` with Unity's main loop through various lifecycle messages (Awake, Start, Update, FixedUpdate, LateUpdate, OnGUI, OnDrawGizmos).

By allowing multiple, distinct **Processing Groups** to be ticked by a single Unity message (e.g., calling `FSM_API.Interaction.Update(groupName)` inside the `Update()` loop), you gain deterministic, ordered control over high-priority and low-priority logic, all while benefiting from the core API's **deferred mutation safety**.

Advanced Workflow: Decoupling Logic from Ticking

The workflow centers on declaring named *Processing Groups* and assigning them to the desired Unity lifecycle events.

1. Configure Processing Groups

The default setup creates groups named after Unity's core loop functions, allowing them to be customized or expanded via the Inspector.

```
// Example: In Awake() of FSM_UnityIntegrationAdvanced  
// A developer can add custom groups to a specific Unity message.  
// The core uses this to map multiple groups (AI, UI, Physics) to one Unity message.  
public void AddProcessingGroup(string unityMessage = "Update", string groupName) {  
    {  
        // ... Logic to add a new group (e.g., "AI_Movement") to the "Update" message.  
    }  
}  
  
// In the Update() function:  
void Update()  
{  
    // FSM_API ticks all defined groups associated with the Update message  
    // ensuring consistent order (e.g., AI_HighPriority runs before AI_LowPriority)  
    foreach (var group in _updateProcessingGroup)  
    {  
        FSM_API.Interaction.Update(group);  
    }  
    FSM_API.Interaction.Update(_unityHandles);  
}
```

2. Define and Instance FSMs

When defining or instancing your FSM, you simply specify which Processing Group it belongs to.

```
// 1. Define your AI FSM.  
FSM_API.Create.CreateFiniteStateMachine("CharacterAI", processRate: 1, processingGroup: "AI")  
// ... Define states and transitions ...
```

```
.BuildDefinition();
```

```
// 2. Create an instance for an in-game character.
```

```
var characterContext = new CharacterContext();
```

```
FSM_API.Create.CreateInstance("CharacterAI", characterContext, "AI_Movemen
```

✨ The Future: Introducing the FSM Layer Packages

This Advanced Integration is specifically designed to work seamlessly with our upcoming **FSM Layer** packages. These extensions will provide:

- **Design Pattern Implementations:** Pre-built, configurable FSM Definitions for common patterns like **State Stacks**, and complex **Behavior Trees**.
- **Game Constructs:** Drop-in solutions for standard game logic:
 - **Player Input Handler FSMs** (e.g., walk -> run -> jump).
 - **UI Navigation FSMs** (e.g., main menu -> options -> credits).
 - **Combat System FSMs** (e.g., idle -> windup -> attack -> recovery).

This transition moves the developer from **creating** foundational logic to merely **configuring** robust, peer-reviewed logical blocks.

Included Demos & Examples

The Advanced package includes the following demos to showcase the flexibility of using the core FSM_API features within the Unity engine:

Demo Name	Description	Concepts Showcased
Simple Light Demo 	A foundational example demonstrating how to use the Finite State Machine (FSM) API to control a simple object's state based on user input.	Core FSM concepts: states , transitions , and context .
Simple Rotation Demo - RotationFSM	Showcases the core strength of the FSM_API philosophy: defining the logic once and reusing it many times with different data contexts, to continuously rotate three separate objects back and forth within a constrained angular range.	Decoupled and reusable state logic, constrained angular range control, separating state logic (RotationFSM) from object data (RotationContext).

Simple Scalar Demo - ScalarFSM	Reinforces the FSM_API core principle of decoupled logic for highly reusable components to autonomously oscillate the scale of three distinct objects (X-axis, Y-axis, and XY-axes).	Decoupled logic for highly reusable components, constrained scaling control, and Multi-axis constraint checking .
Simple Translation Demo - TranslationFSM	Highlights the versatility and reuse potential of a single Finite State Machine (FSM) definition for controlling multiple instances of basic mechanical motion: constrained linear translation .	Versatility and reuse potential of a single FSM definition , controlling constrained linear translation , and boundary checks using the Dot Product .
Simple Rotation, Scalar & Translation Demo - FSM Composition	Illustrates the true power of FSM composition and the decoupling of state logic from the data context, demonstrating how three completely independent FSM instances can all run concurrently on the same physical object without conflict.	FSM Composition and Concurrency by Design by safely operating multiple, concurrent FSMs on the same data, and zero boilerplate duplication .

 Brought to you by

The Singularity Workshop – Tools for the curious, the bold, and the systemically inclined.

Because state shouldn't be a mess.

